

Objectifs

- Mettre en œuvre la BDD du jeu
- Commencer à construire les premières routes API REST.

Cette étape est essentielle pour préparer l'interaction entre le modèle de données et l'application front-end.

Instructions

Voir page 2

Outils pédagogiques et moyens associés

Création de BDD :

- Rappels sur le SQL : [GitHub - detygon/cheatsheet-sql: Aide-mémoire SQL](#) et [GitHub - enochtangg/quick-SQL-cheatsheet: A quick reminder of all SQL queries and examples on how to use them.](#)
- Document de l'ORM sélectionné

ORM recommandés :

(cela varie énormément en fonction du backend)

- JS : Sequelize, Prisma
- Python : Django ORM, SQLAlchemy (SQLAlchemy)

Outils de test API :

- Postman, Bruno ou Insomnia
- Extension Postman sur VSCode
- Extension VSCode HTTP Client:
<https://marketplace.visualstudio.com/items/?itemName=mkloubert.vscode-http-client>
→ Super pour l'utiliser avec des "HTTP files" (cf la doc. de l'extension)

M2 INFO

Coordination Front & Back

Livrable(s)

- Lien vers le repository
- BDD avec données mocks
- Serveur d'API fonctionnelle + 2 routes

Points d'attention

- BDD opérationnelle
- Cohérence entre modèle ORM & BDD
- Respect des conventions REST

Bonus :

- Mettre en place les premiers contrôles d'erreurs simples (ex: 404 ou 500)

Instructions

1. Créer la BDD

- Créer le modèle physique de données (MPD)
- Implémenter le modèle dans un SGBD (moteur de BDD) au choix
- Utiliser directement SQL ou un ORM

2. Insérer quelques données de test

- 2 ou 3 enregistrements minimum par table
- Idéalement mettre en place un système de mock (built-in ou custom)

3. Initialiser un serveur API

- Config. un serveur avec une technologie comme Express.js, Django ou Spring par exemple (selon la stack choisie et validée lors des séance 1 & 2)

4. Créer au moins 2 routes simples sur l'API REST

- Les routes doivent interagir avec la BDD.

5. Tester les routes avec un client de test API